

Date:

Exp No:6

Aim:Implementation of Dining Philosopher's Problem using threads and semaphores.

Theory

The Dining Philosophers Problem is a classical synchronization problem

Five philosophers are sitting around a circular table.

- One chopstick is placed between each pair of philosophers (total 5 chopsticks).
- A philosopher alternates between **thinking** and **eating**.
- To eat, a philosopher must acquire both the left and right chopsticks
- Problems in the System
- Deadlock

If all philosophers pick up their left chopstick at the same time, each will wait for the right chopstick indefinitely, resulting in deadlock.

- Starvation

A philosopher may never get both chopsticks if others continuously eat.

Concepts Used

Threads

Each philosopher is implemented as a separate thread using pthread.

Semaphores

Semaphores are used to control access to shared resources.

- Binary semaphore (mutex) ensures mutual exclusion.
- Each chopstick is represented as a semaphore initialized to 1.

1. Write the c program OS_exp6.c

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
```

```
#define N 5
#define EAT_COUNT 3 // Number of times each philosopher eats
```

```
sem_t chopstick[N];
pthread_t philos[N];
```

```

void *philosopher(void *num)
{
    int id = *(int *)num;

    for(int i = 0; i < EAT_COUNT; i++)
    {
        printf("Philosopher %d is Thinking\n", id);
        sleep(1);

        sem_wait(&chopstick[id]);           // Left chopstick
        sem_wait(&chopstick[(id + 1) % N]); // Right chopstick

        printf("Philosopher %d is Eating (%d)\n", id, i+1);
        sleep(2);

        sem_post(&chopstick[id]);           // Release left
        sem_post(&chopstick[(id + 1) % N]); // Release right
    }

    printf("Philosopher %d has finished eating.\n", id);
    pthread_exit(NULL);
}

int main()
{
    int i;
    int phil_num[N];

    for (i = 0; i < N; i++)
        sem_init(&chopstick[i], 0, 1);

    for (i = 0; i < N; i++)
    {
        phil_num[i] = i;
        pthread_create(&philos[i], NULL, philosopher, &phil_num[i]);
    }

    for (i = 0; i < N; i++)
        pthread_join(philos[i], NULL);

    printf("\nAll philosophers have finished eating.\n");
    return 0;
}

```

2.Compile and Run

```

gcc OS_exp4_1.c
./a.out

```

3..Expected output

```
Philosopher 0 is Thinking
Philosopher 1 is Thinking
Philosopher 3 is Thinking
Philosopher 2 is Thinking
Philosopher 4 is Thinking
Philosopher 0 is Eating (1)
Philosopher 3 is Eating (1)
Philosopher 0 is Thinking
Philosopher 4 is Eating (1)
Philosopher 2 is Eating (1)
Philosopher 3 is Thinking
Philosopher 4 is Thinking
Philosopher 2 is Thinking
Philosopher 1 is Eating (1)
Philosopher 3 is Eating (2)
Philosopher 3 is Thinking
Philosopher 1 is Thinking
Philosopher 2 is Eating (2)
Philosopher 0 is Eating (2)
```

Result

The Dining Philosophers Problem was successfully implemented using threads and semaphores